



Deploying More Projects

One-Box, Many-Sites Automation Guide

One Hetzner server

Many client sites

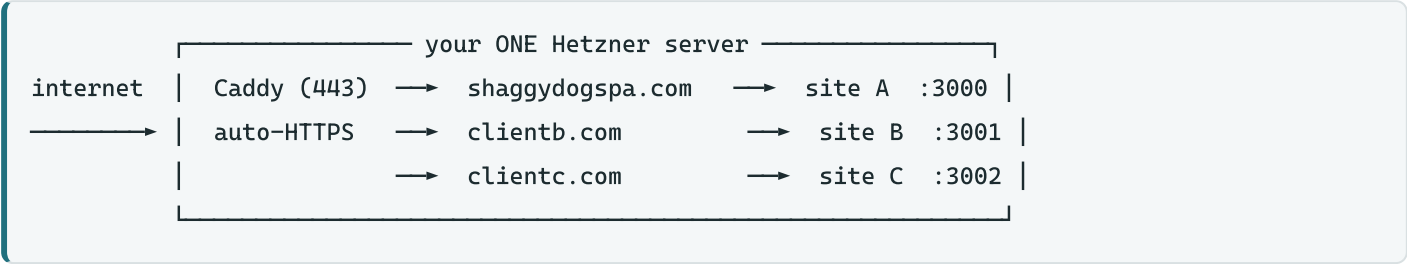
2 scripts

Automatic HTTPS

How to put a brand-new client website on your existing server — in one command.

The idea in one picture

Your server is already set up (Node, Caddy, swap). Adding another website doesn't mean a new server — it means **one command** that clones the project, builds it, and wires up its own address with HTTPS. Every site is fully isolated.



Each site has its own folder, port, service, database, and photos — they never interfere. One flat monthly bill, unlimited low-traffic sites.

The two commands you'll use

When	Command
Add a brand-new site	<code>add-site.sh <name> <repo> <port> <domain></code>
Push an update to a site	<code>SERVICE=<name> deploy.sh</code>

Both live in `~/shaggy-dog-spa/scripts/` on your server and are already installed.

Before you add a site — two things ready

- **The project's GitHub repo URL.** Each client site is its own repo — usually a copy of this one that you rebrand in `/admin` (no coding needed).
- **The client's domain name.** You'll point its DNS at the server at the end.

Add a new site — one command

Log into the server (`ssh deploy@YOUR_SERVER_IP`), then run — filling in the four values:

```
~/shaggy-dog-spa/scripts/add-site.sh clientb https://github.com/you/clientb.git 3001 clientb.c
# | | | |
# | name | the site's GitHub repo |port | its do
```

Pick the values like this:

Value	What to use
name	A short label, e.g. <code>clientb</code> (becomes the folder + service name)
repo	The <code>https://github.com/.../...git</code> URL of that client's site
port	A number no other site uses — <code>3001</code> , then <code>3002</code> , <code>3003</code> ...
domain	The client's domain, e.g. <code>clientb.com</code>

That one command does the whole checklist for you:

1. Clones the repo into `~/sites/<name>`
2. Writes its `.env` (its own database + media folder, unique port)
3. Installs, builds, creates the database, and loads starter content
4. Creates a **systemd service** so it runs 24/7 and auto-restarts
5. Adds its **domain to Caddy** and turns on **automatic HTTPS** 

It **refuses to run** if that name, port, or folder already exists (so you can't break an existing site), and it **checks the Caddy config** before reloading.

The one manual step: DNS

After the command finishes, point the client's domain at your server: at their domain registrar, add two **A records** → your server's IP address:

```
A    @    YOUR_SERVER_IP
A    www  YOUR_SERVER_IP
```

Once DNS propagates (minutes to a couple hours), **https://clientb.com** is live with a valid certificate — Caddy handles it automatically.

Update a site later — one command

When a site's code changes (you or a developer push to its GitHub), deploy the update with:

```
SERVICE=clientb ~/sites/clientb/scripts/deploy.sh
```

It pulls the latest code, reinstalls only if dependencies changed, rebuilds, applies any database migrations, and restarts *that* site — with clear `[1/5]...[5/5]` progress. For the main Shaggy Doggy Spa site, the name defaults to `shaggy` , so it's just `~/shaggy-dog-spa/scripts/deploy.sh` .

Two rules to remember

1. Every site needs a unique port.

3000 is the first site; use 3001, 3002, 3003... for the next ones.
2. Build one site at a time.

The build is memory-heavy — don't run two `add-site.sh` or `deploy.sh` at the same time. Running sites are light; only the build is heavy.

Optional: fully hands-off (push → auto-deploy)

If you want a site to **update itself whenever you push to GitHub** — no command at all — that's **GitHub Actions**. It SSHes into the server and runs `deploy.sh` on every push to `main`. It needs a one-time setup (a deploy key + three GitHub secrets). Ask and it can be wired up per project.

Quick reference card

Goal	Command (on the server)
Add a new client site	<code>~/shaggy-dog-spa/scripts/add-site.sh <name> <repo> <port> <domain></code>
Deploy an update	<code>SERVICE=<name> ~/sites/<name>/scripts/deploy.sh</code>
Update the main site	<code>~/shaggy-dog-spa/scripts/deploy.sh</code>
See a site's logs	<code>journalctl -u <name> -f</code>
Restart a site	<code>sudo systemctl restart <name></code>
List all sites	<code>ls ~/sites</code>

Troubleshooting

Symptom	Fix
"port is already in use"	Pick a different port number (3002, 3003...)
"name already exists"	That site is already set up — use <code>deploy.sh</code> to update it instead
New domain has no HTTPS	Confirm its DNS A records point to the server; <code>journalctl -u caddy -f</code>
Build "out of memory"	You built two at once, or swap is off — build one at a time
Site won't start	<code>systemctl status <name></code> · <code>journalctl -u <name> -f</code>

Companion docs in the repo: `DEPLOY.md` (first-time server setup) and `MULTI-SITE.md` (the manual version of this guide).